📚 جزوه سیستم عامل | فصل ۹: حافظه اصلی

📝 بخش b۳: پشتیبانی سخت‌افزاری صفحه‌بندی و TLB

۱. چالش پیاده‌سازی جدول صفحه (Page Table Issue) 🐢

جدول صفحه (Page Table) یک لیست بزرگ است که می‌گوید هر صفحه کجای حافظه است.

- **مشکل:** این جدول کجاست؟
  - اگر از **ثبات‌ها (Registers)** استفاده کنیم: خیلی سریع است، اما جدول‌های امروزی (با میلیون‌ها صفحه) در ثبات جا نمی‌شوند.
  - اگر در **حافظه اصلی (RAM)** نگه داریم: جا می‌شود، **اما سرعت نصف می‌شود!**
    - چرا؟ چون برای دسترسی به یک متغیر باید:
      1. یک بار به RAM برویم تا جدول صفحه را بخوانیم (تا بفهمیم آدرس فیزیکی کجاست).
      2. یک بار دیگر به RAM برویم تا خودِ داده را بخوانیم.
    - نتیجه: هر دستور ۲ برابر کند می‌شود!
- **راه‌حل:** استفاده از **TLB** (بافر ترجمه در کنار).

۲. بافر ترجمه در کنار (TLB) چیست؟ 🚀

TLB (Translation Look-aside Buffer) یک کش سخت‌افزاری بسیار کوچک و **فوق‌العاده سریع** است که درون پردازنده (CPU) قرار دارد.

کارکرد: آدرس‌های صفحاتی که "اخیراً" استفاده شده‌اند را به همراه آدرس قابشان ذخیره می‌کند.

مراحل ترجمه آدرس با TLB:

1. CPU شماره صفحه (p) را می‌دهد.
2. سخت‌افزار **همزمان** در تمام خانه‌های TLB جستجو می‌کند: "آیا صفحه p را داری؟"
3. **حالت اول: TLB Hit (توفیق):**

- بله، پیدایش کردم! شماره قاب (f) بلافاصله برگردانده می‌شود.
- *هزینه:* تقریباً صفر (چون جزو خط لوله CPU است).

حالت دوم: TLB Miss (شکست):

- خیر، در TLB نیست.
- حالا باید به جدول صفحه در RAM مراجعه کنیم (کُند).
- آدرس قاب را پیدا می‌کنیم و یک کپی از آن را **در TLB می‌نویسیم** تا دفعه بعد سریع باشد.

۳. زمان دسترسی مؤثر (Effective Access Time - EAT) ⏱️

این فرمول در کنکور و امتحانات بسیار مهم است! فرض کنیم:

- زمان دسترسی به حافظه اصلی = m (مثلاً ۱۰ نانوثانیه).
- زمان جستجو در TLB $\approx$ $0.
- نرخ برخورد ($Hit Ratio) = α (درصد دفعاتی که در TLB پیدا می‌شود).

$$EAT = (Hit_Time) \times \alpha + (Miss_Time) \times (1 - \alpha$$

- **زمان Hit:** فقط یک دسترسی به حافظه (برای خود داده) $\rightarrow m$
- **زمان Miss:** یک دسترسی برای جدول صفحه + یک دسترسی برای خود داده $\rightarrow 2m$

$$EAT = m \times \alpha + 2m \times (1 - \alpha)$$$$EAT = 2m - m\alpha$$

- *مثال:* اگر دسترسی حافظه ۱۰ns و نرخ هیت ۸۰٪ باشد:
  - $EAT = 0.8 \times 10 + 0.2 \times 20 = 8 + 4 = 12$ ns
  - یعنی سیستم ۲۰٪ کند شده است. (اگر نرخ هیت ۹۹٪ باشد، کندی فقط ۱٪ خواهد بود).

۴. حفاظت حافظه در صفحه‌بندی (Protection) 🛡️

چون هر آدرس باید از جدول صفحه رد شود، بهترین جا برای کنترل دسترسی همین‌جاست. به هر ردیف جدول صفحه، چند بیت اضافه می‌کنیم:

بیت‌های دسترسی (RWX):

- مشخص می‌کند این صفحه:
  - **Read-only** است؟ (فقط خواندنی، مثل کد برنامه).
  - **Read-Write** است؟ (مثل متغیرها).
  - **Execute-only** است؟
- اگر برنامه بخواهد در یک صفحه Read-only بنویسد، سخت‌افزار تله (Trap) می‌اندازد.

بیت اعتبار (Valid-Invalid Bit): ✅❌

- **v) Valid:** این صفحه در حافظه وجود دارد و قانونی است.
- **i) Invalid:** این صفحه متعلق به فضای آدرس پردازه نیست (یا در دیسک است).
- *مثال:* اگر برنامه ۱۰ صفحه دارد ولی جدول صفحه ما ۱۰۰۰ ردیف دارد، بیت اعتبار برای ۹۹۰ ردیف دیگر Invalid می‌شود تا برنامه نتواند به آن‌ها دسترسی پیدا کند.

۵. صفحات مشترک (Shared Pages) ❤️

صفحه‌بندی اجازه می‌دهد چند پردازه مختلف، به **یک قاب فیزیکی مشترک** اشاره کنند.

- **کاربرد:** اشتراک‌گذاری کدهای کتابخانه (مثل libc) یا کدهای فقط خواندنی (مثل ویرایشگر متن).
- **شرط مهم:** کد باید **Reentrant** (باز-ورود) باشد. یعنی در حین اجرا خودش را تغییر ندهد (متغیرهایش جدا باشند، اما دستوراتش یکی باشند).
- **نتیجه:** صرفه‌جویی عظیم در حافظه (یک نسخه از libc در حافظه است ولی ۴۰ تا برنامه از آن استفاده می‌کنند).

📊 جدول جمع‌بندی نکات کلیدی

مفهوم	توضیح	نکته تستی
TLB	کش سریع برای جدول صفحه.	اگر نباشد، سرعت حافظه نصف می‌شود.
ASID	شناسه پردازه در TLB.	اجازه می‌دهد TLB را هنگام تعویض پردازه خالی نکنیم (Flush نشود).
Valid Bit	بیت اعتبار در جدول صفحه.	دسترسی‌های غیرمجاز (خارج از محدوده) را می‌گیرد.
Reentrant	کدی که تغییر نمی‌کند.	شرط لازم برای اشتراک‌گذاری صفحات کد.

Code